

Lab Assignment

Task 1: Configure the Web App Manifest

Create a valid `manifest.json` file in your application root directory. Define the application's identity by including the following properties: `name`, `short_name`, `start_url`, `display` (set to `standalone`), `theme_color`, and an `icons` array containing paths to icons of at least 192x192px and 512x512px sizes. Link this manifest file in the `<head>` section of your `index.html` file.

Task 2: Register the Service Worker

Write a script in your main client-side JavaScript file to register your service worker. Wrap the registration code inside a check for feature support (`'serviceWorker'` in `navigator`) and ensure it registers the `sw.js` file safely after the window has fully loaded to prevent blocking the initial page paint.

Task 3: Implement Static Caching on Install

Inside your `sw.js` file, add an event listener for the install lifecycle event. Create a cache named `v1-static-assets` using the Cache Storage API, and use `event.waitUntil()` along with `cache.addAll()` to pre-cache your core application shell files, including your main `index.html`, basic CSS stylesheets, core JavaScript files, and an offline fallback page.

Task 4: Clean Up Old Caches on Activate

Add an event listener for the activate lifecycle event inside your `sw.js` file. Use this event to clear out older versions of your cache. Write logic that loops through all existing cache keys, checks if they match your current cache name, and uses `caches.delete(key)` to drop old caches so that updated assets take effect immediately.

Task 5: Intercept Network Requests for Offline Fallback

Add a fetch event listener to your `sw.js` file to intercept outgoing network requests. Implement a "Cache, falling back to Network" strategy for your static assets. If a request fails completely because the browser is offline, catch the network error and serve your pre-cached offline fallback page instead.